00 · META

# Chalk Book Guide

Purpose, structure & conventions

A personal technical chalk book. After a session of problem-solving, building or learning with an AI, the work is distilled into one calm, skimmable note that can be revisited later for a fast refresher, on a small local site or as a PDF.

This is the read-only copy of `CLAUDE.md`, which is the source of truth.

Local — your machine | Remote — the server | Config file | Concept | AWS Console | Gotcha | Best practice | Paths

## Purpose

- ✓ **Capture** recent work while it is fresh: commands, configs, decisions, mistakes.
- ✓ **Revisit** it later and get the gist in under a minute.
- ✓ **Repeat** it cold, months later, without re-researching anything.
- ✓ **Trace** what was done, per project and date, so a future AI can recall which scenarios have been worked through.

### Two kinds of knowledge

**Recurring**

Used again and again: `systemctl`, ssh config, naming rules.

**The note is a fast reference.** Patterns, quick tables, the one flag you always forget.

**One-off**

Set up once, or a rare fix: growing an EBS volume, a logrotate permission error.

**The note is a complete recipe.** Every step in order, every flag explained, so it can be redone from zero.

**Best practice** · When in doubt, write for the forgetful future self. Assume nothing is remembered.

## The collection

```
chalk-book/
├── CLAUDE.md                    # this guide (source of truth)
├── 00-chalk-book-guide.pdf      # read-only copy of this guide
├── chalk-pastel.css             # the note colour palette (user-owned)
├── index.html                   # the site: open it in a browser
├── site/                        # catalogue (notes.js), menu, search, dark theme
├── notes/                       # one HTML file per note
│   ├── tinker-town-01-aws-ssh-firewall.html …
│   ├── note-layout.css          # shared layout and components
│   └── fonts/                   # bundled Inter, renders offline
├── archive/                     # original PDFs from before the HTML rebuild
└── .claude/skills/memory-dump/  # the note-making workflow
```

### File naming

```
<project>-<NN>-<topic>.html
```

| PART | RULE | EXAMPLE |
|---|---|---|
| **project** | Slug from the project list, so the name says where the work happened. | `tinker-town` |
| **NN** | Two digits, counting up within each project. | `03` |
| **topic** | 2–4 words naming the subject, not the outcome. | `cron-logrotate` |
| **scope** | One topic per file. A long session becomes several notes. | — |
| **date** | Inside the note and in `site/notes.js`, not in the file name. | `2026-08-30` |
| **00-** | Reserved for meta files about the chalk book itself. | `00-chalk-book-guide` |

### Projects

| SLUG | DISPLAY NAME | WHAT IT IS |
|---|---|---|
| `tinker-town` | Project Tinker Town | Self-paced AWS Linux sysadmin lab, a one-week crash course in 7 modules |
| `pokedex` | Pokédex | The Pokédex web project |
| `meta` | Chalk Book | Notes about the chalk book itself |

A new project gets a row here and in `CHALK_PROJECTS` in `site/notes.js`, which can also hold its roadmap of modules.

### Note types

**Walkthrough**
What was done in a session, start to finish. Most notes are this type.

**Reference**
A cross-cutting rulebook other notes depend on, such as naming conventions. It says so in its summary.

## Anatomy of a note

Every note follows the same top-to-bottom shape, so the eye always knows where to look.

1. **Header band**: eyebrow chip `PROJECT TINKER TOWN · 03 · MODULE 1 · 2026-08-30`, title words coloured blue, pink, green, yellow in turn, a short subtitle, and a 1–2 sentence past-tense summary.
2. **Legend**: colour chips for every block type used in the note.
3. **What we covered**: a ✓ checklist of outcomes, skimmable on its own.
4. **Full command sequence**: steps under short headings, each tagged Local or Remote
5. **Supporting sections**: concepts, diagrams, field tables, quick-reference tables.
6. **Bugs hit & fixed**: one card per real error, **Cause → Fix**.
7. **Where things live** and **Best practices** (short cards) as needed.
8. **Part of** line: the project, with links to related notes through the site.
9. **Footer** on printed pages: `Tinker Town 03 — Systemd Service · 1 / 3`

### Block types

| BLOCK | PALETTE COLOUR | USED FOR |
|---|---|---|
| **Local** | `--sky` | Commands run on your own machine |
| **Remote** | `--lav` | Commands run on the server |
| **Config file** | `--butter` | File contents, with the path as the block title |
| **Concept** | `--peri` | The *why* behind something; command output |
| **Gotcha** | `--peach` | Traps that bit, or nearly did; bug cards |
| **Best practice** | `--mint` | The recommended way, and why |
| **Paths** | `--pink` | File and folder locations; tables; title accent |
| **AWS Console** | `--coral` | Steps done in a web console, not a shell |

Inline callouts use a bold label and a dot: **Gotcha ·** **Concept ·** **Key rule ·** **Naming note ·** **Updated rule ·**

## Writing style

- ✓ **Skim first.** Headings, checklists and tables carry the story; prose only fills gaps.
- ✓ **Short.** Sentences under ~20 words. No intro paragraphs, no filler.
- ✓ **Every command gets a comment** explaining what it does and what each flag means.
- ✓ **Config files stay clean.** Annotations sit beside the file or in a field table, never inside it.
- ✓ **Real by default, except access, location and private life.** Actual commands, paths and error messages, but: no keys, passwords or tokens; placeholders such as `<instance-ip>` for public IPs, account IDs and emails; nothing about the user's private life. Every personal admin account, live or test, is written as `adm-kiwi`; the steps stay the same.
- ✓ **Keep the mistakes.** Errors and their fixes are often the most valuable part.
- ✓ **Respect history.** When a convention changes, mark old names as legacy instead of rewriting earlier notes.
- ✓ **Calm tone.** Plain, friendly and factual. No hype, no emoji.

```
sudo mkdir -p /mnt/data    # -p: create parents, no error if it exists
```

**Gotcha ·** Many config formats break on inline comments. That is why config blocks are never annotated inside the block itself.

## Visual style

**Key rule ·** `chalk-pastel.css` is the palette for every note, until the user says otherwise. Colours are used only through its variables, never as hex values. The file belongs to the user.

### How the palette is built

| PART | EXAMPLE | USED FOR |
|---|---|---|
| **chalk** | `--lav` | Chips, borders, left bars |
| **-bg** | `--lav-bg` | Box tints |
| **-ink** | `--lav-ink` | Coloured text, comments |
| **-chip** | `--lav-chip` | Text on a chalk chip |

**Concept** · Notes link the palette instead of copying it. Changing `chalk-pastel.css` restyles every note at its next render; PDFs keep their look until re-rendered.

Layout lives in `notes/note-layout.css`: A4 on `--paper`, an `--asphalt` header band, Inter for text, DejaVu Sans Mono for code, soft tinted blocks with a coloured left bar.

### The site

- ✓ `index.html` opens straight from disk, no build step. Each note shows as a pastel A4 sheet in a dark frame in the warm greys of Claude's dark mode (`site/site-theme.css`, kept apart from the note palette).
- ✓ Left menu: search on top, then notes per project with roadmap progress and planned modules. The ☰ button (three chalk sticks) collapses it to a rail of note numbers and colour bars, with a chalk-stick tile for search.
- ✓ Search reads the full text of every note; results replace the list while typing, each note once with its hit count, and opening one highlights every hit. Rebuild the index with `node site/build-index.mjs`.
- ✓ The persistent main footer links Home, Backlog and Complete Tasks (private pages in `.prog/`, never committed) and Contact.
- ✓ Right panel: recently viewed notes, newest first. The name is one shared element, `<chalk-wordmark>`.
- ✓ **Export** opens a small menu; for now it holds **Save as PDF**, which opens the browser's print dialog for the open note.
- ✓ The menu starts as a rail; the menu button opens and closes it. Recently viewed is a rail too, sliding open over the note in 0.3 s while hovered and closing as soon as the cursor leaves.

## How notes are made & changed

**Concept** · The notes are **read-only by design**. Nobody edits a note in a viewer; every change goes through an AI session.

### Adding a note: the memory-dump skill

1. Work on something, in any project folder.
2. While it is fresh, say **“memory-dump”** or **“chalk that up”**. A personal pointer skill makes this work in every folder. An AI may also offer a dump when the topic changes or about an hour of content has built up.
3. The skill plans the notes, fills the template in `notes/`, adds them to `site/notes.js`, rebuilds the search index, checks every page, then commits and offers to push.
4. Check the note on the site and ask for any changes in the same session.

### Changing a note or this guide

- ✓ Ask for the change in a session. The AI edits the HTML source or `CLAUDE.md`.
- ✓ After `CLAUDE.md` changes, this guide's HTML and PDF are updated so all three stay in sync.
- ✓ Note PDFs are made on demand from the site; the originals from before the HTML rebuild are kept in `archive/`.

Local **Rendering**

```
.claude/skills/memory-dump/render.sh notes/<name>.html [out-dir]   # → <name>.pdf
```

## Rules for AI assistants

- ✓ Read `CLAUDE.md` before creating or changing anything in the chalk book.
- ✓ Use the `memory-dump` skill to make notes, and `chalk-pastel.css` for every colour.
- ✓ Match the existing notes' structure and look exactly; open one as a reference if unsure.
- ✓ Never edit, rename or delete an existing note or archived PDF unless the user asks.
- ✓ Keep `CLAUDE.md`, this guide's HTML and its PDF in sync, and change them only when the user asks.
- ✓ Keep private data out *before* it is committed: GitHub may still serve a pushed commit after a force-push. If something slips through, tell the user; don't rewrite history without asking.
- ✓ Commit locally as needed, but push only when the user asks.